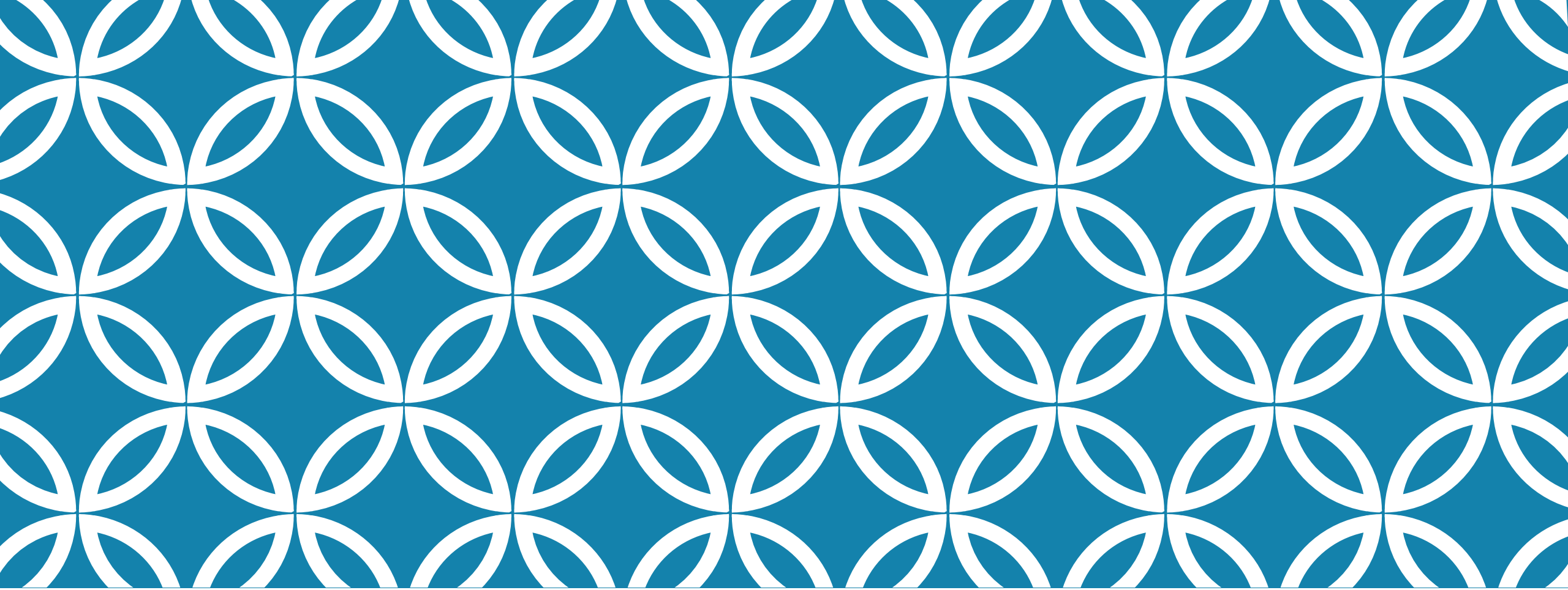


GÖRSEL PROGRAMLAMA-II

Öğr. Gör. Erhan AKAGÜNDÜZ

GİRİŞ

- Bir yazılım geliştiricisi, kod satırları arttıkça işlerin karmaşıklaştığını ve kodun yönetilmesinin zorlaştığını fark eder.
- Tekrarlanan kod blokları ve hatalar, kodun okunabilirliğini ve geliştirilebilirliğini olumsuz etkiler.
- Bu sorunları çözmek için **fonksiyonlar** veya **metotlar** kullanılır.
- Fonksiyonlar ve metotlar, belirli işlemleri yaparak tekrar eden kod yazma yükünü azaltır, kodun okunabilirliğini artırır ve hataların yönetimini kolaylaştırır.
- Ayrıca, kodun tek bir yerden yönetilmesi, geliştirilmesini ve bakımını daha kolay hale getirir.

METOT OLUŞTURMA KURALLARI

- **İsim:** Metotlar isimlendirilmelidir ve değişken kurallarına uymalıdır.
- **Erişim Seviyesi:** Metotların erişim belirleyicileri olabilir (public, private, protected). Erişim belirleyici belirtilmemişse varsayılan olarak private'tır.
  - **Private:** Sadece tanımlandığı sınıftan erişilebilir.
  - **Public:** Her yerden erişilebilir.
  - **Protected:** Sınıftan ve türetilmiş sınıflardan erişilebilir.

METOT OLUŞTURMA KURALLARI

- **Niteleyici (Opsiyonel):**
  - static, abstract, virtual, sealed gibi niteleyiciler kullanılabilir.
  - Static metotlar bellekte sabitlenir ve performans düşebilir.
- **Dönüş Tipi:**
  - Metot bir sonuç döndürecekse, dönecek veri türü belirlenir (int, string, vb.).
  - Değer döndürmeyen metotlar için **void** kullanılır.

METOT OLUŞTURMA KURALLARI

- **Metot Adı:**
  - Metodu çağırmak için kullanılan isimdir.
- **Parametre Listesi (Opsiyonel):**
  - Metotlar parametre alabilir.
  - Parametreler, tip adı ve değişken adı ile tanımlanır ve virgül ile ayrılır.
  - Parametreler, metot çağırıldığında işlem yapmak için kullanılır.

MAİN() METODU

- **Main() Metodu:**
  - Programın başlangıç noktasıdır ve genellikle sadece bir tane **Main()** metodu bulunur.
- Genellikle void olarak tanımlanır, ancak bazı durumlarda **int** türünde bir geri dönüş değeri de kullanabilir.
- Bu geri dönüş değeri, işletim sistemine programın bitiş durumu hakkında bilgi sağlar.
- **Main()** metodu bazen parametre de alabilir.

ÖZEL İFADELER

- **Return Anahtar Sözcüğü:**
  - Geri değer döndüren metotlarda kullanılır ve metot içindeki işlemlerin ardından belirlenen dönüş tipinde değer döndürülmesini sağlar.
- **This Anahtar Sözcüğü:**
  - This anahtar sözcüğü, metot içinde sınıfın üyeleri ile yerel değişkenler arasında ayrım yapmayı sağlar.
  - Global değişkenlere referans verir ve benzer adlara sahip değişkenlerde karışıklığı önler.

ÖZEL İFADELER

- **This Kullanımına Örnek:** Örnekte **thisKullanimi** adında oluşturulan **metot** parametresi olan **deger** ile global olarak tanımlanan **deger** değişkeninin isimlerinin aynı olduğuna dikkat ediniz.
- İşte bu benzerliğin ayrımını yapabilmek ve global olarak tanımlanmış değer değişkeninin kullanılmasını sağlamak amacıyla **"this"** anahtar sözcüğü koda eklenmiştir.
- İşlemlerin ardından sonuç 50 olarak çıkmıştır.

```
public partial class Form2 : Form
{
    int deger = 25;
    1 başvuru
    public Form2()
    {
        InitializeComponent();
    }

    1 başvuru
    public int thisKullanimi(int deger)
    {
        this.deger = this.deger * deger;
        return this.deger;
    }

    1 başvuru
    private void button1_Click(object sender, EventArgs e)
    {
        MessageBox.Show("Sonuç = " + thisKullanimi(2).ToString());
    }
}
```

PARAMETRE VE ARGÜMAN

- **Parametre:**
  - Metot oluşturulurken tanımlanan değişkenlerdir. Metotlar, bu parametreler aracılığıyla dışarıdan veri alabilir.
- **Argüman:**
  - Metot çağrılırken verilen değerlerdir.
  - Bu değerler, metot içindeki parametrelere atanır.
- Parametreler metotların kodlarına dışarıdan veri geçişini sağlar.
- Böylece metot içindeki değişkenler çağrıldıkları yerden değer alabilir ve değiştirilir.

METOT TÜRLERİ

- **Metotlar,** değer döndürme durumlarına göre ikiye ayrılır ve ardından parametre alıp almamalarına göre tekrar sınıflandırılır.
- **Geriye Değer Döndürmeyen Metotlar:** Bu metotlar, herhangi bir sonuç döndürmez, sadece belirli işlemleri gerçekleştirirler. Bu metotlar bir değişkene atanamaz.
- **Örneğin:**
  - **MessageBox.Show():** Kullanıcıya mesaj gösterir.
  - **Clear():** Temizleme işlemi yapar.
- Bu metotların geri dönüş tipi **void** olarak tanımlanır.

METOT TÜRLERİ

- **Parametre Almayan ve Değer Döndürmeyen Metotlar:**
  - Bu metotlar, parametre listesinde herhangi bir parametre tanımlaması yapılmadan oluşturulur ve çalışmaları için argüman gerektirmezler.
  - Örneğin, **Clear()** metodu, *parametre almaz* ve *değer döndürmez*.

METOT TÜRLERİ

- **Parametre Alan ve Değer Döndürmeyen Metotlar:**
  - Bu metotlar, parametre listesinde parametre tanımlaması yapılmış ve işlemlerini gerçekleştirmek için argüman gerektiren metotlardır.
  - Parametre verilmediğinde, genellikle bir **sözdizimi (syntax)** hatası oluşturur.
  - Örneğin, **MessageBox.Show()** metodu, parametre olarak aldığı bilgiyi kullanıcıya bilgi kutusunda gösterir ve herhangi bir değer döndürmez.

METOT TÜRLERİ

- **Geriye Değer Döndüren Metotlar:**
- Geriye değer döndüren metotlar, yapılan işlemlerin ardından belirli bir değeri geri verir.
- Bu metotlar, geri dönüş tipine uygun bir değişkene atanabilirler ve **return** anahtar kelimesi ile bir değer döndürürler.
- Metodun geri dönüş tipi, **string, double, float** gibi herhangi bir tipte olabilir.
- Metotların kodları, return ifadesi ve geri dönüş tipiyle uyumlu bir değer ile bitirilmelidir.

REF VE OUT ANAHTAR SÖZCÜKLERİ

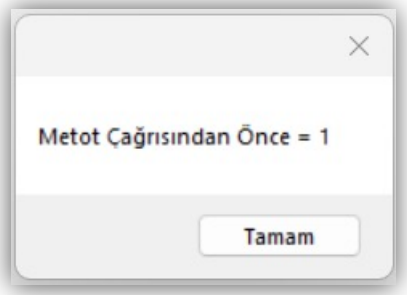
- C# dilinde, metotlara argüman olarak verilen değer tipleri ve referans tiplerinin kopyaları üzerinden işlem yapılır.
- Ancak, bazen argümanın orijinal değerinin değişmesi istenebilir.
- Bu durumda, değişkenin referans adresi ile işlem yapılmalıdır.
  - **ref:** Metodun çağrıldığı yerden değişkenin referansını alarak, değişkenin orijinal değerini değiştirmeyi sağlar.
  - **out:** Metotların birden fazla değer döndürmesine imkan tanır. Bu anahtar kelime ile de değişkenin referans adresi kullanılarak orijinal değer değiştirilebilir.

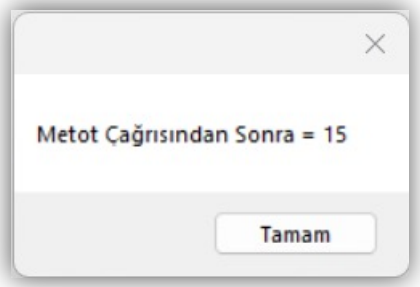
REF VE OUT ANAHTAR SÖZCÜKLERİ

- Bunun için ref anahtar sözcüğünü hem metot tanımında, hem de metot çağrısında kullanmamız gerekiyor.

```
public int refDegerAtama(ref int deger)
{
    deger = 15;
    return deger;
}
1 başvuru
private void button1_Click(object sender, EventArgs e)
{
    int sayi = 1;
    MessageBox.Show("Metot Çağrısından Önce = "+sayi.ToString());

    refDegerAtama(ref sayi);
    MessageBox.Show("Metot Çağrısından Sonra = " + sayi.ToString());
}
```





REF VE OUT ANAHTAR SÖZCÜKLERİ

- ref ve out kelimelerinin farkı argüman olarak verilecek değişkenlere atama yapılmış olup olmamasıdır.
- **ref** anahtar sözcüğünün kullanıldığı metotlara verilecek argümanların değerinin atanmış olması gerekir.
- Ancak **out** anahtar sözcüğü için değişkene ilk değer ataması yapmak zorunlu değildir.

PARAMS ANAHTAR SÖZCÜĞÜ

- **params** anahtar sözcüğü, metotlarda belirli bir sayıda parametre yerine, aynı türdeki ***belirsiz sayıda parametreyi*** işlemek için kullanılır.
- **params** ile tanımlanan parametreler genellikle bir dizi olarak alınır, ancak dizinin geleneksel yapısından farklı olarak işlenir.
- **params** parametresi, metot tanımında en son parametre olmalıdır çünkü kendisinden sonra gelen parametreleri kabul etmez.
- Bir metotta yalnızca bir **params** anahtar sözcüğü kullanılabilir.

PARAMS ANAHTAR SÖZCÜĞÜ

```
public int paramsEnBuyukSayiyiBul(params int[] sayilar)
{
    int en_buyuk_sayi = -1;

    if (sayilar.Length > 0)
    {
        en_buyuk_sayi = sayilar[0];
    }

    for (int i = 1; i < sayilar.Length; i++)
    {
        if (en_buyuk_sayi < sayilar[i])
        {
            en_buyuk_sayi = sayilar[i];
        }
    }
    return en_buyuk_sayi;
}
1 başvuru
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("En Büyük Sayı = " + paramsEnBuyukSayiyiBul
      (7,10,3,5,9));
}
```

